

Caltrack / QuickTIMS — Testing Team Handover Document

1. Product Overview & Architecture

1.1 Summary

Caltrack (internally referenced as QuickTIMS) is a multi-tenant workforce management SaaS platform. It enables organizations to manage employee onboarding, time tracking (with GPS, geofencing, and photo/selfie verification), shift scheduling, field task execution, leaves/holidays, inventory tracking, mileage logs with multi-jurisdictional reimbursement calculations, and payroll processing.

1.2 Tech Stack

- **Frontend:** React 19 + Vite. Styled using vanilla CSS variables. State managed with Redux Toolkit. Routing is handled via React Router v6. Map layers are rendered using Leaflet / React Leaflet. Icons are provided by Lucide React.
- **Backend:** Python 3.13 + Django 5.2. Web APIs are constructed using Django REST Framework (DRF) with custom permission layers. WebSockets alerts use Django Channels (ASGI/Daphne).
- **Database:** Multi-tenant setup managed by [django_tenants](#). Local development utilizes SQLite (with tenant middlewares bypassed), and production environments use PostgreSQL with separate schemas dynamically isolated per tenant (domain mapped). Databases contain collections representing models mapped through the Django ORM.

1.3 Tenant Isolation & Routing

- **Public Schema:** Houses shared data such as [User](#) accounts, [Company](#) registrations, [Domain](#) routing maps, and global compliance [Region](#) records.
- **Tenant Schema:** Dynamically created on organization registration (e.g., [org_name](#)). Houses all operational tables (e.g., [Employee](#), [TimeLog](#), [Shift](#), [Task](#), [LeaveRequest](#), [PayrollRecord](#), [MileageTrip](#)).
- **Route Guards:**
 - **RequireAdmin** (in [App.jsx](#)): Restricts access to routes such as Reports, Scheduling, Payroll, Compliance, and Approvals to users with [admin](#) or [manager](#) roles.
 - **RequireAdminSettings** (in [App.jsx](#)): Redirects standard employees away from administrative settings pages, allowing them only to access their personal Profile settings.

2. Authentication, Authorization & Security

2.1 JWT Cookie-Based Authentication

- **Flow:** User authenticates via username/password or Google OAuth.
- **Cookies:** Access token stored in httpOnly cookie [qt_access](#) (valid for [/](#), lifetimes configurable, default 8 hours). Refresh token stored in httpOnly cookie [qt_refresh](#) (restricted to [/api/auth/refresh/](#), default 7 days).
- **Token Refreshing:** Client triggers [POST /api/auth/refresh/](#) automatically. It reads [qt_access](#) and [qt_refresh](#) cookies and rotates the access cookie via [RefreshView](#).
- **Logout:** Triggers [POST /api/auth/logout/](#) which clears both cookies from the browser via [LogoutView](#).

2.2 Google OAuth Onboarding

- **Flow:** Google login verifies Google access tokens via Google UserInfo API in [GoogleLoginView](#).
- **Auto-Onboarding:** If the user email does not exist in the database, it automatically generates a user account, maps them to the default/demo company schema, and creates an [Employee](#) profile record with a generated sequential employee ID.

2.3 Registration & Invitation Accepts

- **Organization Signup:** Multi-step register view via [RegisterView](#). Creates a new [Company](#) and [Domain](#) in the public schema, triggers tenant schema creation, creates the admin user account, and links them.
- **Team Invites:** Admins send invites which generate tokenized links. Invitee submits password and name via [AcceptInviteView](#). The system performs a tenant lookup across all schemas, creates the new user inside the organization's tenant space, and changes the invite status to **accepted**.

2.4 Multi-Factor Authentication (2FA)

- **Setup:** Authenticated users activate TOTP via [TwoFactorSetupView](#). System generates a secure base-32 key, creates a provisioning URI, and renders a QR code.
- **Verification:** User enters the TOTP code to verify. If valid, the system activates **two_fa_enabled = True** and returns 8 single-use hex backup codes.
- **Removal:** Requires the user's password to disable 2FA.

2.5 SMS/OTP Gateway & Security

- **Endpoints:** [SendOTPView](#) and [VerifyOTPView](#).
- **Security Mechanisms:**

1. **IP Rate Limiting:** Max 5 OTP requests per IP per minute. 2. **Phone Rate Limiting:** Max 5 OTP requests per phone number per hour. 3. **Resend Cooldown:** Enforces a strict 30-second wait between successive requests for the same phone number. 4. **OTP Hashing:** The 4-digit code is encrypted using SHA-256 with the Django **SECRET_KEY** before caching. 5. **Expiries:** Cached OTPs are valid for 5 minutes. 6. **Failed Attempts Block:** If the user enters incorrect codes 5 times, the cached OTP is deleted, and the user must request a new code.

- **Twilio & Console Fallback:** Real SMS delivery is attempted via Twilio. In local development or if credentials are empty, the code falls back to printing the OTP code directly inside the server console log.

2.6 Password Recovery & Identity Verification

- **Verification:** [PasswordResetVerifyIdentityView](#) accepts a username, email, or employee ID. It scans the public database, then loops through all tenant schemas to identify the profile, returning masked contact info (e.g., **su*@company.com**) to prevent enumeration.
 - **Recovery Request:** Generates a token (via Django's **PasswordResetTokenGenerator**) and a base64-encoded user ID in [PasswordResetRequestView](#). Sends a recovery email containing the secure activation link.
 - **Confirmation:** Validates the token and updates the password in [PasswordResetConfirmView](#). Returns the employee ID on success.
-

3. Onboarding & Employee Verification (Activation Journey)

The **Activation Journey** is a high-fidelity 5-step onboarding wizard for newly onboarded technicians/employees ([ActivationJourneyPage.jsx](#)).

3.1 Step 1: Registration & Selfie Capture

- **Contact Information:** Captures full name, email, phone, and address.
- **Mobile Verification:** Calls the secure OTP endpoint (`apiSendOTP` and `apiVerifyOTP` mapped from [authService.js](#)).
- **Selfie Capture:** Requires the webcam to capture a selfie.
- **Face Detection Check:** Integrates client-side face detection using [faceVerify.js](#).
- **Biometrics Scan:** Runs a mock biometric scanner simulation to record the user's face pattern.

3.2 Step 2: Document Verification

- **Document Collection:** Technicians submit IDs and bank details:
 - Aadhaar Card ID & PDF upload
 - PAN Card ID & PDF upload
 - Bank Account Number, IFSC code & Passbook PDF upload
- **OCR Simulation:** Runs a simulated verification sequence logging OCR progress, extracting text, calculating confidence scores, and verifying details against Step 1 registration inputs.
- **Dynamic Visual Generation:** Dynamically creates base64-encoded visual SVGs mimicking Aadhaar, PAN, and Bank passbooks containing the technician's name and captured profile picture.

3.3 Step 3: Video Training Academy

- **Video Watch Session:** Technicians watch a mandatory training video.
- **Interactive Quiz:** At the end of the video, a quiz is unlocked:
 - *Question:* "What is Caltrack's primary workforce intelligence methodology?"
 - *Correct Answer:* "B) Real-time AI-driven operational telemetry"
- Must pass the quiz to unlock the next step.

3.4 Step 4: WebRTC Interview Check

- Simulates a video call check-in to confirm compliance.
- Presents interactive dialog questions to select from (confirming years of experience, safety compliance, and SLA policies).

3.5 Step 5: Admin Review Clearance

- Technician waits on a screen indicating pending verification.
 - **Real-time Polling:** Frontend polls `/api/auth/registration-dossier/` (which reads/writes `caltrack_activation_dossier.json` on the server via [RegistrationDossierView](#)) every 1.5 seconds to capture the administrator's review decision.
 - **Decisions:**
 - *Approved:* Triggers a full-screen holographic celebration and redirects the technician to the active workspace.
 - *Rejected:* Displays a rejection dialog stating the admin's reasons and allows re-submitting.
-

4. Time Tracking & Geofencing Engine

4.1 Geofence Configuration

Locations ([Location](#) model) represent zones, offices, or job sites. They carry coordinates ([lat](#), [lng](#)) and are defined by one of three geofence types: 1. **Circle**: Evaluates coordinates against a circular boundary defined by [geofence_radius](#) (meters, defaults to company-wide radius, typically 200m). 2. **Polygon**: Evaluates coordinates against a GeoJSON polygon structure (stored in [geofence_polygon](#)). 3. **Hybrid**: Allowed if the coordinate falls inside the circle **OR** inside the polygon.

4.2 The Geofence Evaluation Engine ([geofence_service.py](#))

Whenever an employee clocks in or checks status, the system runs [evaluate\(\)](#) to determine compliance.

```
mermaid graph TD
  A[Start Clock-in/Validate] --> B{Geofencing Enabled globally?}
  B -- No --> C[Allow clock-in, geofence_passed = True, mode = off]
  B -- Yes --> D[Resolve Candidate Locations]
  D --> E{Any candidates available?}
  E -- No --> F[Allow clock-in, geofence_passed = True, mode = off]
  E -- Yes --> G{GPS coordinates provided?}
  G -- No --> H[Allow clock-in, geofence_passed = True, distance = None]
  G -- Yes --> I[Find closest location candidate]
  I --> J{Evaluate geofence bounds: Circle/Polygon/Hybrid}
  J -- Inside --> K[Allow clock-in, geofence_passed = True]
  J -- Outside --> L{Shift location constraint mismatch?}
  L -- Yes --> M[Apply Shift Enforcement Mode]
  L -- No --> N[Apply Geofence Enforcement Mode]
  M & N --> O{Enforcement Mode?}
  O -- off/warn --> P[Allow clock-in, geofence_passed = False]
  O -- block --> Q{Admin Override used?}
  Q -- Yes --> R[Allow clock-in, geofence_passed = True, admin_override_used = True]
  Q -- No --> S[Block clock-in, return 403 error]
```

candidate resolution precedence: 1. Explicit list passed to the validator. 2. Active [Shift](#) location constraint (if a shift specifies a location, that is the only candidate). 3. [allow_all_locations = True](#) override on [Employee](#) (allows all active locations in the company). 4. Permitted locations set via [EmployeeLocation](#) mappings. 5. Legacy [JobSite](#) assigned to the employee (wrapped using [_JobSiteLocationAdapter](#) with circle geofence rules).

Boundary Check Mathematics ([geo_utils.py](#)):

- **Haversine Distance**: [haversine_m\(\)](#) computes great-circle distance in meters between coordinate pairs.
- **Ray-Casting Algorithm**: [point_in_ring\(\)](#) checks if a point lies within a polygon. Projects a horizontal ray to the right of the point and counts intersections with the polygon's edges. An odd number of crossings indicates the point is inside the polygon.
- **Bounding Box Sweep**: [point_in_bbox\(\)](#) pre-computes the bounding box of a polygon as a quick reject filter before running the Ray-Casting calculation.
- **Centroid Math**: [ring_centroid\(\)](#) calculates the average coordinate of all polygon vertices to determine the location's center point.
- **Distance to Edge**: [distance_to_ring_m\(\)](#) computes the shortest distance from the user's coordinate to any edge segment of the polygon using vector math.

4.3 Shift-Location Constraints

If the employee is scheduled for a [Shift](#) that requires a specific location, and they attempt to clock in at a different location (or outside the required location's bounds), the system flags a **Shift Mismatch**.

4.4 Enforcement Modes

Company settings ([shift_enforcement_mode](#) and global settings on [Company](#)) govern the outcome if an employee is outside the boundary:

- **off**: Clock-in allowed. Geofence flags are set to [geofence_passed = False](#) and [admin_override_used = False](#).

- **warn**: Clock-in allowed. Bypasses blocking but flags a warning.
- **block**: Clock-in is rejected, returning a **403 Forbidden** response detailing how far away the employee is. Bypassed only if an administrator is requesting an override (`admin_override_used = True`).

4.5 Additional Geofencing Endpoints

- **Validate Point** (`/api/time/geofence/validate-point/` mapped to [GeofenceValidatePointView](#)): A dry-run validation endpoint used by the frontend to provide real-time distance feedback to the employee on the map before they attempt to clock in.
 - **Location Map Overview Caching**: Polled by admins via [LocationOverviewView](#). The response is memoized using local memory cache (`locmem`) for 30 seconds per tenant. Can be bypassed using a `?fresh=1` query parameter.
 - **WebSocket Alerts**: Successfully clocking in or out broadcasts real-time status alerts (online/offline) to the admin live tracking panels.
-

5. Field Worker Task Management

5.1 Task Life Cycle

1. **Creation**: [Task](#) is created by an admin with a category, priority, description, location, required tools/spares, and optional verification rules (selfie, before/after photos). 2. **Acceptance**: Newly assigned tasks default to `pending_acceptance`. The worker must explicitly accept the task (updating it to `accepted`) or decline it, providing a reason. Declining alerts the administrator for reassignment. 3. **In Progress**: When the worker starts the task, the status updates to `in_progress` and it links to their active [TimeLog](#). If they are not clocked in, starting the task clocks them in. 4. **Suspension**: A worker can suspend a task (providing a reason). While suspended, they can accept a **Gap Job** (a short, nearby task). When the gap job is completed, they resume the parent task. The system calculates cumulative active seconds, excluding the suspended time. 5. **Completion**: The task is marked `completed`. If it is linked to a time log, completing the task clocks the worker out.

5.2 Travel & Journey tracking

Tasks support sub-statuses mapping the physical journey to a client site: `None` → `on_the_way` (tap Start Travel) → `reached_site` (tap I've Arrived, stores `reached_site_at`) → `working` (tap Start Work, updates task status to `in_progress` and stores `work_started_at`) → `done` (completed).

5.3 Billed Hours Rounding (Sub-1-Hour Rules)

On task completion, if `billed_hours` is not explicitly set, the system calculates it based on the elapsed time between `started_at` and `completed_at` using two rules defined in [compute_billed_hours](#): 1. **Under 1-Hour Estimate** (If `estimated_hours` < 1.0):

- Actual duration ≤ 15 minutes → billed hours = **0.50 hours**.
- Actual duration ≤ 45 minutes → billed hours = **1.00 hour**.
- Actual duration > 45 minutes → billed hours = actual hours rounded up to the nearest 0.5 hours (e.g., 50 minutes becomes 1.0 hour; 75 minutes becomes 1.5 hours).

2. **Standard Estimate** (If `estimated_hours` ≥ 1.0):

- Billed hours = actual elapsed hours (rounded to 2 decimal places).

5.4 Face Verification

If `require_selfie` is activated, workers must upload photos at task start and completion. The system processes the image using face verification utils, calculating a similarity score and setting `face_match_status` to `verified` or `failed`.

6. Mileage Tracking & Reimbursement

Calculates travel expense reimbursements based on distance traveled using local regulatory rates.

6.1 Regulatory Reimbursement Formulas

Reimbursements are calculated by [calculate_trip_reimbursement](#) in [services.py](#) according to the selected jurisdiction:

- **United States (IRS Rules):**

Reimbursement = Distance (miles) × rate_per_mile_usd (default: \$0.67/mile)

- **India Rules:**

Reimbursement = Distance (km) × rate_per_km_inr (default: ₹3.50/km)

- **United Kingdom (HMRC Rules):**

Calculates mileage totals relative to a threshold of 10,000 miles per UK tax year (spanning April 6 to April 5 of the next year):

- For Year-To-Date (YTD) mileage **under 10,000 miles**: uses the high rate (default **£0.45/mile**).
- For YTD mileage **above 10,000 miles**: uses the low rate (default **£0.25/mile**).
- **Threshold Splitting**: If a trip crosses the 10,000-mile mark, it is split: the portion under 10,000 is paid at £0.45, and the portion above is paid at £0.25. The low-rate portion is flagged as a taxable benefit: `is_taxable_excess = True` and the taxable amount is stored in `taxable_excess_amount`.

6.2 Auto-Approvals

Trips are automatically approved if the distance in miles is ≤ `auto_approve_below_miles`.

6.3 Automation Hooks

- **From Tasks**: Automatically generates travel trips between consecutive tasks (ordered by start times) using the tasks' latitude and longitude via [create_trip_from_task_sequence](#).
- **From Inventory**: Automatically generates mileage trips for inventory transfers between locations with coordinate data via [create_trip_from_transfer](#).
- **Paid Travel Hours**: Can automatically generate a paid [TimeLog](#) for the travel duration, assuming an average speed of 50 km/h via [create_travel_timelog](#).

7. Compliance & Audit Trails

7.1 Immutable Audit Logging

- **Trigger**: Database signals in [signals.py](#) capture all additions, edits, and deletions on the [TimeLog](#) table.
- **Context Capture**: An [AuditRequestMiddleware](#) stores the request in thread-local storage, allowing database signals to record the modifying user's ID and IP address.
- **Immutability**: Once saved, audit log records cannot be edited or deleted. An application-level constraint blocks updates in [AuditLog.save](#).
- **Retention**: Saves a `retention_until` date calculated as 3 years from the creation date (supporting DOL/WTR audits).

7.2 Overtime Alerts

Monitors timesheets to trigger real-time alerts on the admin dashboard ([OvertimeAlert](#) model):

- **US FLSA**: Weekly hours approaching 40 hours, or exceeded 40 hours.
- **California/Alaska Daily OT**: Shift hours exceeding 8 hours (standard daily OT) or 12 hours (double time).

- **UK WTR:** Weekly hours approaching 48 hours, or the 17-week rolling average exceeding 48 hours.

7.3 UK Right-to-Work Tracking

Monitors verification documents (Passports, BRPs, Share Codes) in [RightToWork](#). Calculates expiry dates and flags automated reminders when documents are within 60 days, 30 days, or 7 days of expiry.

7.4 UK WTR 48-Hour Opt-Out Agreements

Tracks signed employee agreements to opt out of the WTR 48-hour limit in [WTROptOut](#). If an opt-out is not active and the employee's 17-week rolling average exceeds 48 hours, the system flags a compliance violation.

8. Payroll Engine

8.1 Region-Specific Compliance Rules

The payroll engine supports regional calculations determined by the employee's location, resolved via [resolve_region](#) and loaded via [get_compliance_rules](#):

Parameter	United States (FLSA)	United Kingdom (WTR / PAYE)
---	---	---
Standard Hours	40 hours / week	37.5 hours / week
Weekly Overtime	\$1.5\times\$ rate for hours \$> 40\$	\$1.0\times\$ rate (typically no OT pay)
Daily Overtime	California: \$1.5\times\$ for hours \$> 8\$, \$2.0\times\$ for hours \$> 12\$ Alaska: \$1.5\times\$ for hours \$> 8\$	None
Paid Leave	Hours added to regular rate	Included in calculations
Taxes	Managed via state tax configurations	PAYE Income Tax: brackets at 20%, 40%, 45% after Personal Allowance (£12,570) resolved via calculate_uk_income_tax_annual
Insurance	Deducted according to local settings	National Insurance: employee (8% / 2%) and employer (13.8%) calculations via calculate_uk_ni_annual
Holiday Accrual	None	WTR Holiday Accrual: 12.07% of hours worked, split into Reg 13 (4 weeks) and Reg 13A (1.6 weeks) pots via calculate_uk_holiday_accrual

8.2 FLSA Exempt Status Bypass

Employees marked as **Exempt** are excluded from overtime calculations. All worked hours are processed as regular hours.

8.3 UK Rolled-Up Holiday Pay

If enabled for an employee, the system adds a **12.07%** holiday pay premium directly to their gross pay on each payslip, rather than letting hours accrue in holiday pots.

8.4 Monthly Salary-Based Payroll

Admins can run monthly payroll calculations ([DynamicPayrollGenerateView](#)) using country-specific rule structures ([PayrollRule](#)):

- **Gross Salary:** Calculated as $\text{\$}\{Basic\} + \{HRA\}$.
- **Deductions:** Calculated as $\text{\$}\{PF\} + \{ESI\}$.
- **Net Salary:** Calculated as $\text{\$}\{Gross Salary\} - \{Deductions\} + \{Mileage Reimbursement\}$.

8.5 Payroll Invoicing

Admins can generate and email payroll invoices to employees via `send_invoice_email` in [PayrollRecordViewSet](#). If a local SMTP server is not configured, the system logs the rendered HTML invoice directly to the console output.

9. Testing Checklist for QA

To assist the testing team, verify the following test cases for each module:

Auth & Security

- Verify rate limiting on [SendOTPView](#) by making 6 rapid requests from the same IP (verify 429 response on 6th request).
- Verify that entering an incorrect OTP code 5 times deletes the key from the cache and blocks further attempts inside [VerifyOTPView](#).
- Test password reset with a valid username, email, and employee ID; check that [PasswordResetVerifyIdentityView](#) masks the email correctly.
- Verify that 2FA QR code generation works and the backup codes list is returned on successful validation via [TwoFactorSetupView](#).

Onboarding (Activation Journey)

- Test the selfie capture verification; ensure validation blocks users if no face is detected by the client-side webcam logic inside [ActivationJourneyPage.jsx](#).
- Verify that submitting invalid formats for Aadhaar/PAN cards triggers simulated OCR failures and reduces the OCR confidence scores.
- Verify that the video quiz restricts the user from proceeding to step 4 unless the correct option ("B") is selected.
- Perform a mock admin approval change on a registration dossier and verify the page updates within 1.5 seconds without a page refresh.

Time Tracking & Geofencing

- Clock in inside a circle geofence and verify `geofence_passed = True` and `distance_from_site_meters` matches coordinates.
- Attempt to clock in outside a circle geofence with `shift_enforcement_mode = block`. Verify a 403 response is returned showing the correct distance.
- Verify that checking `admin_override = True` allows a clock-in from outside the boundary, logging `admin_override_used = True` in the database.
- Test polygon geofencing by clocking in inside a custom GeoJSON polygon boundary; verify successful validation.
- Verify that checking in during a shift with a location constraint blocks clock-in if the user is at a different location.
- Test that the `validate-point` endpoint returns distance calculations without writing a new log entry to the database.

Task Management

- Verify task state transitions: `pending_acceptance` → `accepted` → `in_progress` → `completed`.
- Test the sub-1-hour billing rounding rule for a task with `estimated_hours = 0.5`:
 - Set elapsed time to 12 minutes; verify `billed_hours = 0.50`.
 - Set elapsed time to 35 minutes; verify `billed_hours = 1.00`.
 - Set elapsed time to 50 minutes; verify `billed_hours = 1.00` (ceil to nearest 0.5).
- Test suspending a task: verify the task status changes to `suspended` and the timer stops tracking active seconds. Start a gap job, complete it, and verify that resuming the parent task resumes the active seconds count.

Mileage & Reimbursement

- Verify US mileage reimbursement uses the flat rate of $\$0.67/\text{mile}$.
- Verify UK mileage reimbursement calculations:

- Set YTD miles to 9,990 and log a 20-mile trip. Verify the trip is split: 10 miles paid at £0.45, 10 miles paid at £0.25, and `is_taxable_excess` is set to `True` for the 10 miles paid at the lower rate.
- Verify that inventory transfers between two locations with coordinates automatically generate a pending mileage trip.

Compliance & Payroll

- Confirm that editing an employee's timesheet entry generates a new record in the immutable [AuditLog](#) containing before/after state snapshots.
- Verify that attempting to edit or delete a record in the [AuditLog](#) table returns an error.
- Verify California payroll calculation rules:
 - Log a 13-hour workday. Verify that 8 hours are calculated as regular, 4 hours as overtime ($\$1.5 \times \$$), and 1 hour as double time ($\$2 \times \$$).
- Test the UK rolled-up holiday pay calculation: verify that the gross pay is increased by **12.07%** when the toggle is enabled on the employee's profile.